IT360 – Information Assurance and Security Course

Project 6

Image Steganography System

Deliverables – Week 1 & 2

Part 1 – Core Concepts, Main Components & Functional Flow
Part 2 – Overview of Main Existing Solutions

Submitted by: Sarah Mneja, Maryem Ammar, Ayoub El Atri
Academic Year: 2025–2026
Professor: Manel Abdelkader

Part 1 – Core Concepts, Main Components & Functional Flow

1.1 Core Concepts:

1.1.1 Image Steganography

Steganography is the practice of concealing a secret message within a non-secret carrier medium so that the very existence of the message remains hidden.

The word originates from the Greek steganos (covered) and graphia (writing).

Steganography aims for perfect invisibility: a third party observing the communication should not even suspect that a secret exists.

Image steganography is the process of hiding information inside an image by modifying its pixel values in a way that is not noticeable to the human eye.

Key distinction from encryption:

Criteria	Steganography	Encryption
Purpose	Hide existence of data	Hide content of data
Visibility	Invisible	Visible but unreadable
Security	Low alone	Strong
Best Practice	Combined with encryption	Used alone or combined

1.1.2 Core Terminology

The following terms are fundamental and will appear throughout this document and the codebase:

Term	Definition
Cover image	The original, innocent-looking host image into which the secret is embedded (e.g. a landscape photograph).
Payload / secret	The data to be hidden: text, binary file, another image, etc.
Stego image	The output image that contains the hidden payload. Visually identical to the cover image.
Stego key	An optional password or cryptographic key that controls how/where the payload is embedded and is required for extraction.
Embedding capacity	The maximum amount of data that can be hidden in the cover image without introducing perceptible distortion.
Imperceptibility	The degree to which the stego image is visually indistinguishable from the cover image.
Steganalysis	The art and science of detecting steganographic content in a suspected carrier. It is the adversarial counterpart of steganography.

1.1.3 The Three Core Properties (Security Triangle)

Any steganographic system must balance three competing properties. Improving one typically degrades the others:

- **Imperceptibility:** The stego image must not reveal any visible or statistical traces of the hidden data. A human observer — and ideally an automated detector — must be unable to distinguish it from the cover image.
- **Capacity:** The system must be able to carry a sufficient amount of hidden data relative to the size of the cover image. LSB techniques typically allow embedding 1 to 3 bits per pixel per channel.
- **Robustness:** The hidden data must survive any transformations that the image might undergo during transmission (e.g. format conversion, mild compression, resizing). Note: lossless formats like PNG must be used with LSB, since JPEG compression destroys the least-significant bit plane entirely.

1.2 Main Components of the System

1.2.1 User Interface (UI)

This is how the user interacts with the system.

- **Role:**

  Upload an image

  Enter secret message
  Choose encode/decode
  Display/download results
- **Can be:** Web page or CLI

1.2.2 Input Handler

This component processes user inputs.

- **Role:**

  Read image file
  Read text message
  Validate inputs (size, format, etc.)

1.2.3 Encoding Module

This is the core part for hiding data.

- **Role:**

  Convert message → binary
  Embed binary into image pixels using LSB
  Generate stego-image

1.2.4 Decoding Module

This retrieves the hidden message.

- **Role:**

  Extract LSB bits from image
  Convert binary → text
  Output the secret message

1.2.5 Image Processing Module

Handles manipulation of the image.

- **Role:**

  Access pixel values
  Modify RGB values
  Save new image
- **Usually done using:** Python libraries (PIL / OpenCV)

1.2.6 (Optional) Security Layer

This makes your project stand out

- **Role:**

  Encrypt message before hiding
  Decrypt after extraction
  Add password protection

1.2.7 Output Module

Handles results.

- **Role:**

  Provide encoded image for download
  Display decoded message

1.3 The LSB Algorithm in Detail

The Least Significant Bit (LSB) method is one of the simplest and most commonly used techniques in image steganography.
Digital images are composed of pixels, and each pixel contains color values (Red, Green, Blue), typically represented as 8-bit binary numbers.
The LSB technique works by modifying the last bit of each pixel value, which has minimal impact on the overall image appearance.

1.3.1 Embedding a message bit into a pixel (example)

Original pixel value (binary):
10110010
Modified pixel value:
10110011
This small change is visually undetectable but allows storage of secret data.

This process is repeated across the R, G, and B channels of successive pixels, giving a capacity of 3 bits per pixel for a standard colour image.

1.3.2 Capacity calculation

For a cover image of W x H pixels in RGB format, the maximum embedding capacity with 1-bit LSB substitution is: **Capacity = W x H x 3 bits**. A 512x512 RGB image can therefore hide up to 786,432 bits = 98,304 bytes = approximately 96 KB of data.

1.3.3 The role of the stego key

Without a key, LSB embedding inserts bits sequentially starting from the top-left pixel. This is trivially detectable by chi-square analysis on the LSB plane. By using the key to generate a random permutation of pixel positions (via a PRNG), the modified pixels are scattered uniformly across the image, defeating most basic statistical detection methods. The recipient must supply the same key to reconstruct the same permutation and read the bits in the correct order.

1.4 Functional Flow

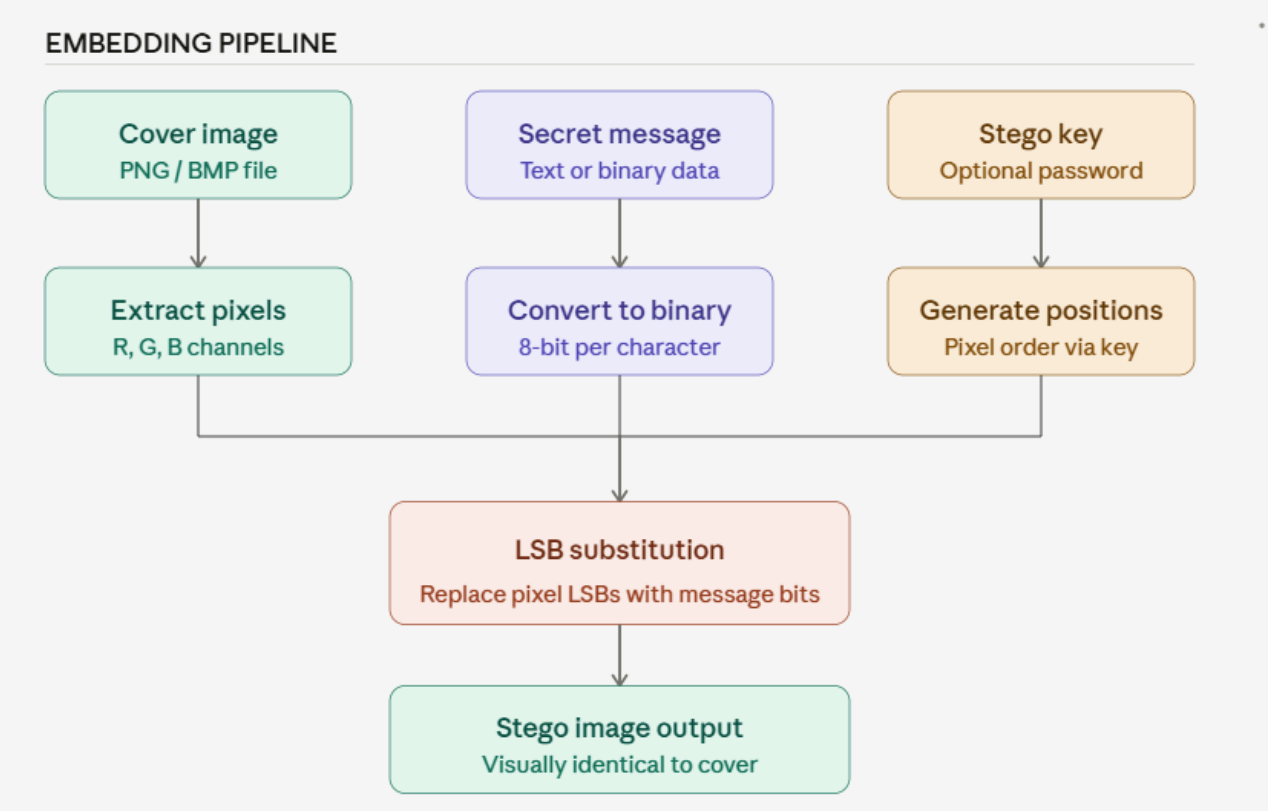
The system operates in two distinct phases: the Embedding Phase (performed by the Sender) and the Extraction Phase (performed by the Receiver). The stego image is transmitted over an insecure channel between the two phases.

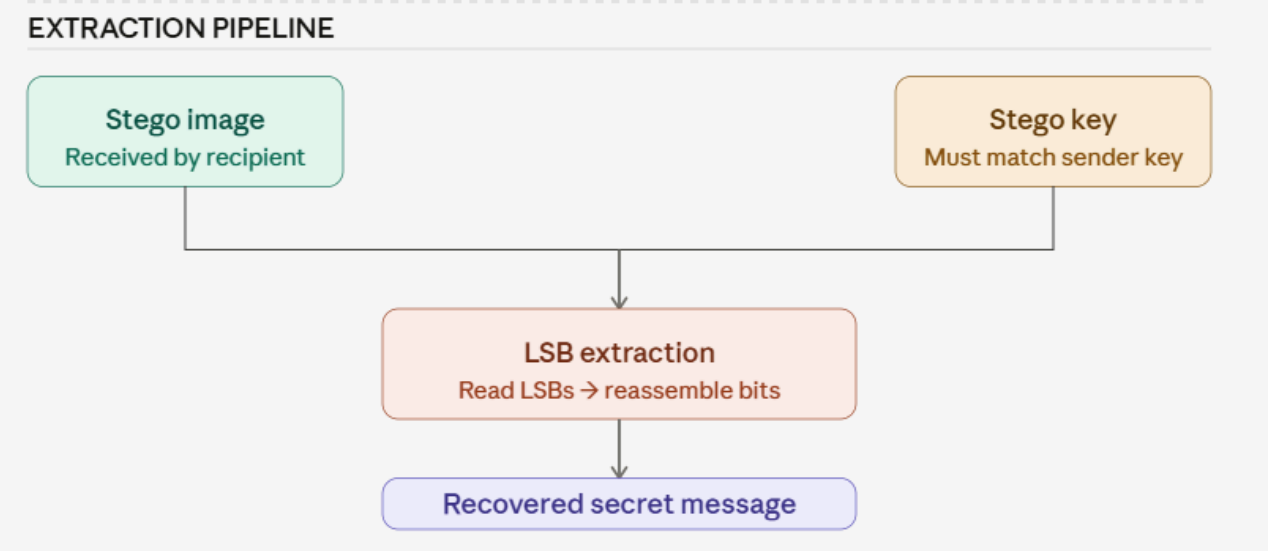
1.4.1 Embedding Phase

- The sender opens the web application and uploads an image (preferably PNG or BMP).
- The sender types the secret message.
- The sender can optionally set a password (stego key).
- The system sends the image, message, and password to the backend.
- The system generates a secure key from the password and uses it to control how the data is hidden.
- The message is converted into binary form, and a special ending marker is added.
- The system hides the message inside the image using the LSB technique, possibly in a shuffled pixel order for better security.
- The system checks image quality (using metrics like PSNR and SSIM).
- The final stego image is generated and returned to the sender for download.
- The sender sends this image to the receiver (via email, messaging apps, etc.), while the password is shared separately for security.

1.4.2 Extraction Phase

- The receiver opens the web application and uploads the stego image.
- The receiver enters the password provided by the sender.
- The system sends the image and password to the backend.
- The system regenerates the same key using the password.
- The system reads the hidden bits from the image in the correct order.
- The binary data is converted back into text.
- The original secret message is displayed to the receiver.


EMBEDDING PIPELINE
Cover image
PNG / BMP file
Secret message
Text or binary data
Stego key
Optional password
Extract pixels
R, G, B channels
Convert to binary
8-bit per character
Generate positions
Pixel order via key
LSB substitution
Replace pixel LSBs with message bits
Stego image output
Visually identical to cover


EXTRACTION PIPELINE
Stego image
Received by recipient
Stego key
Must match sender key
LSB extraction
Read LSBs → reassemble bits
Recovered secret message

Part 2 – Overview of the Main Existing Solutions

2.1 OpenStego

OpenStego is an open-source tool used for image steganography and watermarking.

1. Advantages:

- Easy to use
- Free and open-source
- Supports basic data hiding

2. Disadvantages:

- Limited advanced features
- Basic user interface

2.2 StegHide

Steghide is a command-line based steganography tool that supports embedding data in images and audio files.

1. Advantages:

- Strong embedding techniques
- Supports encryption
- Efficient compression

2. Disadvantages:

- Not beginner-friendly (CLI-based)
- No graphical interface

2.3 SilentEye

SilentEye is a graphical steganography tool designed for ease of use.

1. Advantages:

- User-friendly interface
- Supports plugins
- Suitable for beginners

2. Disadvantages:

- Limited file format support
- Less flexible for customization

2.4 Comparison Summary

Tool	Interface	Strength	Limitation
OpenStego	GUI	Easy to use	Limited features
Steghide	CLI	Powerful & secure	Hard to use
SilentEye	GUI	User-friendly	Less flexible

2.5 Proposed Solution

The proposed system aims to:

- Provide a simple and intuitive interface
- Implement LSB-based image steganography
- Allow encoding and decoding of secret messages
- Optionally integrate encryption for improved security

End of document